Distributed Operating Systems

Election Algorithms

1. **A Bully Algorithm**
2. **A Ring Algorithm**

ELECTIONS

- There are at least two basic strategies by which a distributed system can adapt to failures.
  - operate continuously as failures occur and are repaired
  - The second alternative is to temporarily halt normal operation and to take some time out to reorganize the system.
    - The reorganization of the system is managed by a single node called the coordinator.
    - So as a first step in any reorganization, the operating or active nodes must elect a coordinator.

ELECTION AND SYNCHRONIZATION

- Similar
  - Like Synchronization, all processors must come to an agreement about who enters the critical region (i.e. who is the leader)
- Different
  - The election protocol must properly deal with the case of a coordinator failing. On the other hand, mutual exclusion algorithms assume that the process in the critical region (i.e., the coordinator) will not fail.
  - A new coordinator must inform all active nodes that it is the coordinator. In a mutual exclusion algorithm, the nodes not in the critical region have no need to know what node is in the region.

ELECTION ALGORITHMS

- The two classical election algorithms by Garcia-Molina
  - Bully Algorithm
  - Invitation Algorithm
- Ring Algorithm

The Bully Algorithm

- Assumptions
  - Each node has access to some permanent storage that survives node failures.
  - There are no transmission errors.
  - The communication subsystem does not fail

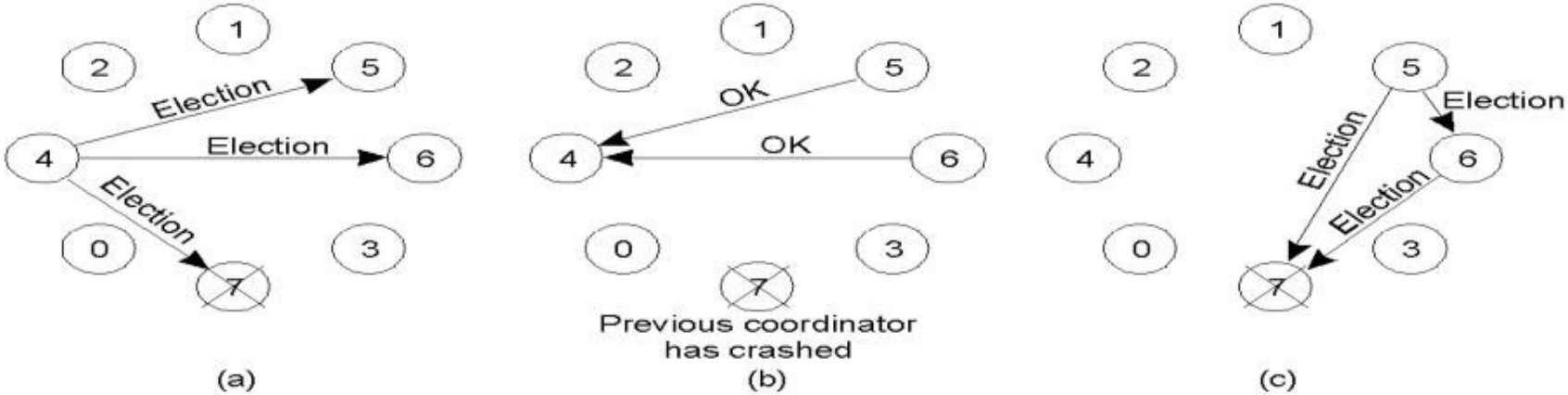
The Bully Algorithm

- State Information For Election
  - Status
    - Down,Election,Reorganization,Normal
  - Co-ordinator
    - The Current co-ordinator of the node
  - Definition
    - The State Information of the task being performed - the application algorithms, list of the participating nodes

The Bully Algorithm

- Each process has an associated priority (weight). The process with the highest priority should always be elected as the coordinator.
- How do we find the heaviest process?
- Any process can just start an election by sending an election message to all other processes
- If a process *P*heavy receives an election message from a lighter process *P*light, it sends a take-over message to *P*light. *P*light is out of the race.
- If a process doesn't get a take-over message back, it wins, and sends a victory message to all other processes.

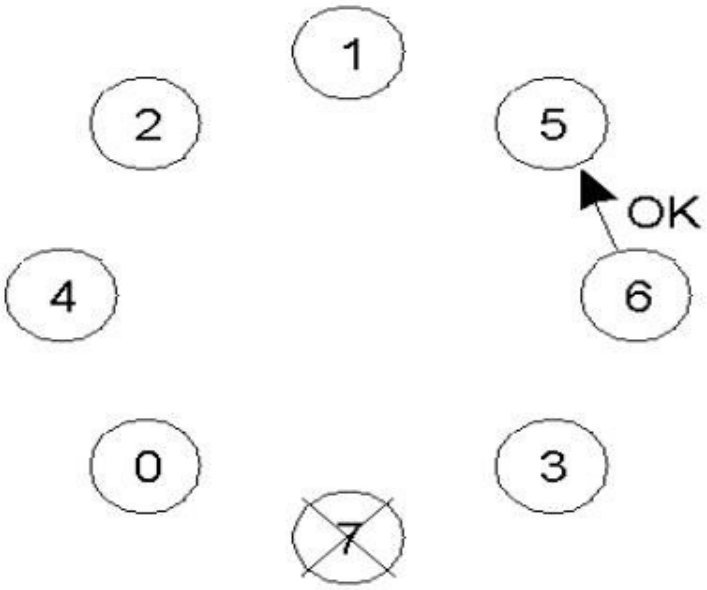
The Bully Algorithm



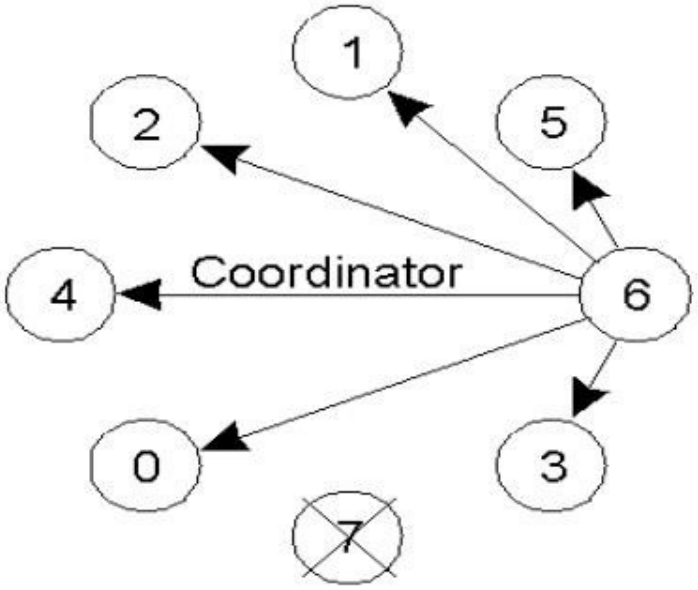
The bully election algorithm

Process 4 holds an election
Process 5 and 6 respond, telling 4 to stop
Now 5 and 6 each hold an election

- Process 6 tells 5 to stop
- Process 6 wins and tells everyone



(d)



(e)

Bully Algorithm: Properties

- Assume n processes initially
  - Worst Case:
    - Smallest process initiates election
    - Requires O(n2) messages
  - Best Case:
    - Eventual leader initiates election
    - Requires (n-1) messages

The Invitation Algorithm

- Bullying algorithm fails for asynchronous systems
- The invitation algorithm classifies nodes into ‘groups’ and elects a co-ordinator for every group
- An additional state variable is indroduced for this asynchronous election
  - Group Number
    - Identifier of the group the processor belongs to

The Invitation Algorithm

- When Processor p wishes to establish itself as the new co-ordinator , it sends out invitations to the potential participants
- Groups that can communicate try to coalesce into larger groups
- Periodically the group co-ordinator searches for other groups

The Invitation Algorithm

- Starting a new Group
  - If a node does not hear from its co-ordinator for a long time, the node declares its own group
  - It could start sending invitations to all the other participants of the old group

A Comparison of the two Algorithms

- Logical Structure
  - Bully –
    - Simple and Efficient because it imposes a logical structure on processors
  - Invitation –
    - Might take a long time for groups to merge because of no strongly defined structure

A Comparison of the two Algorithms

- Synchronous Vs Asynchronous
  - Bully
    - Makes very strong use of bounded response time which are mostly unrealistic
  - Invitation
    - Works correctly in the presence of timing failures

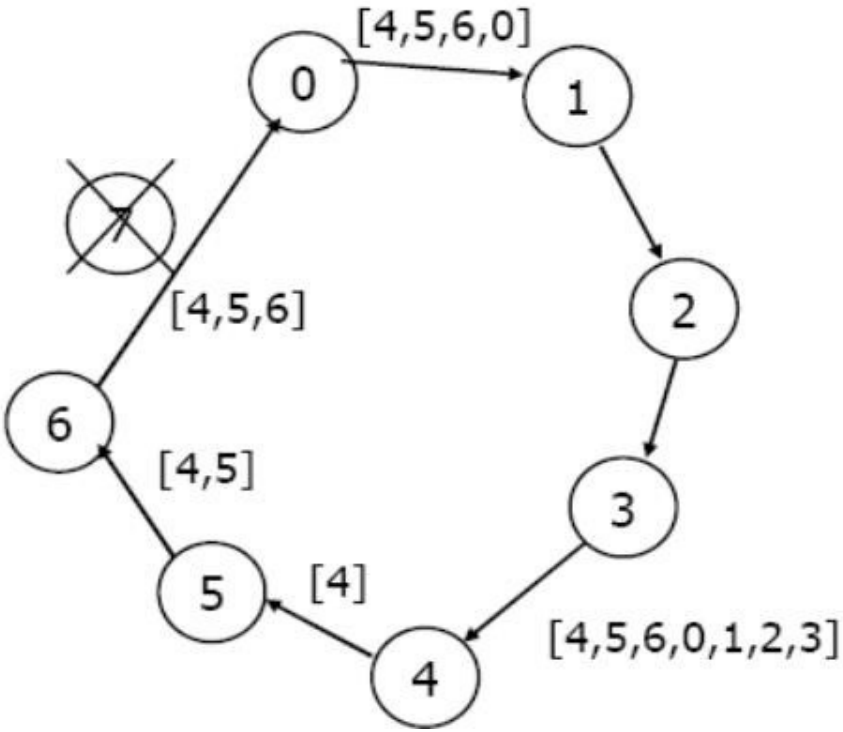
Election in a Ring

- Process priority is obtained by organizing processes into a (logical) ring. Process with the highest priority should be elected as coordinator.

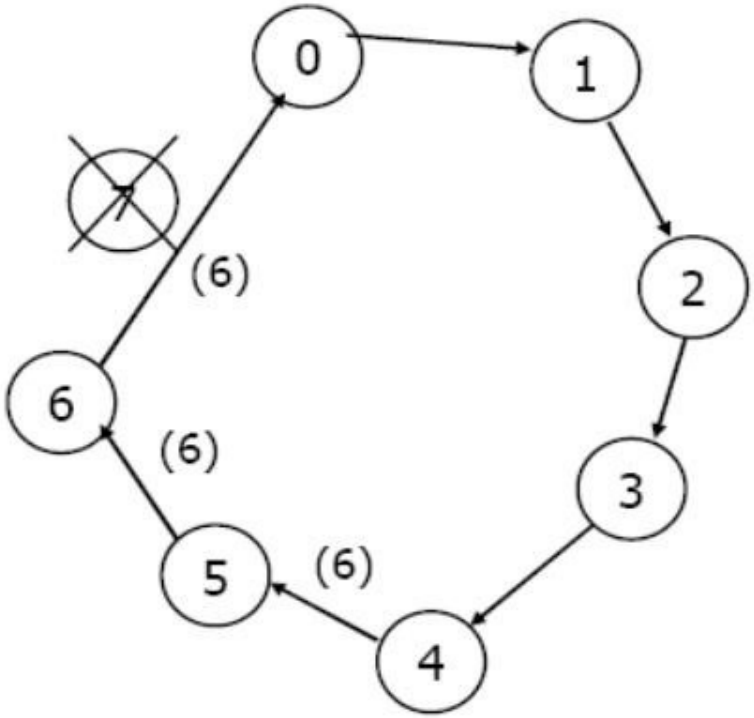
- Each process has a successor
  - Initiation:
    - A process sends an ELECTION message to its successor (or next alive process) with its ID
    - Each process adds its own ID and forwards the ELECTION message

  - Leader Election:
    - Message comes back to initiator
    - Initiator announces the winner by sending another message around the ring

Ring Algorithm: Initiation



Ring Algorithm - Election



Ring Algorithm: Properties

- If only 1 process initiates election:
  - Requires 2n messages
- Two or more processes might simultaneously initiate elections
  - Still ensures election of the same leader
  - Results in extra messages

Thank You